Prelectura: Fundamentos de R para lingüística de corpus

Módulo de Análisis Exploratorio de Datos, Doctorado en Lingüística. Basado en Gries, *Quantitative Corpus Linguistics with R* (Routledge, 2009), capítulo 3.

Las sesiones del módulo están dedicadas al análisis de datos, no a la sintaxis de R. Esta lectura cubre el mínimo necesario para seguirlas. No hace falta memorizar nada: hace falta haber ejecutado el código y resuelto las actividades. Instala primero R (cran.r-project.org) y luego RStudio (posit.co/download/rstudio-desktop). Escribe los ejemplos a mano en un guion de código (*script*) propio; en los bloques de código, las líneas que comienzan con `#>` muestran la salida (*output*) que R devuelve y no deben escribirse.

1. Nociones centrales

R evalúa expresiones y guarda resultados mediante asignación con `<-`. Los nombres distinguen mayúsculas de minúsculas (*case-sensitive*): `corpus` y `Corpus` son objetos distintos.

```
frecuencia <- 42
frecuencia
#> [1] 42
```

Los tipos de datos (*data types*) básicos son `numeric` (números), `character` (texto, siempre entre comillas) y `logical` (`TRUE` o `FALSE`, en mayúsculas y sin comillas). La función `class()` indica el tipo de un objeto. Sin comillas, R interpreta el texto como el nombre de un objeto y falla si no existe.

Las funciones reciben argumentos (*arguments*), por posición o por nombre, y muchos tienen un valor predeterminado (*default*): `round(3.14159, digits = 2)` devuelve 3.14; `round(3.14159)` devuelve 3, porque `digits` vale 0 de forma predeterminada. La ayuda de cualquier función se consulta con `?round`; la sección *Arguments* de la página de ayuda es la que más se usa.

Para definir funciones propias:

```
frec_norm <- function(f, tam_corpus, base = 1000) {
  (f / tam_corpus) * base
}
frec_norm(340, 85000)
#> [1] 4
```

La función devuelve la última expresión evaluada. El argumento `base` puede omitirse porque tiene un valor predeterminado.

Por último, R trabaja situado en un directorio de trabajo (*working directory*), que `getwd()` muestra. En lugar de cambiarlo con `setwd()`, crea un proyecto de RStudio (File, New Project): las rutas quedan

ancladas a la carpeta del proyecto y el código funciona en cualquier computador.

Actividades.

1.1 Crea tres objetos: tu apellido (`character`), el año de inicio de tu doctorado (`numeric`) y si tu investigación usa corpus (`logical`). Verifica el tipo de cada uno.

1.2 Escribe una función `ttr()` que reciba número de tipos léxicos (*types*) y de ocurrencias (*tokens*) y devuelva su cociente. Pruébala con 450 tipos y 1200 ocurrencias.

1.3 Consulta `?mean` y anota para qué sirve el argumento `na.rm`.

2. Vectores

Un vector es una secuencia de elementos del mismo tipo y se crea con `c()`. Si se mezclan tipos, R convierte todo al más general sin avisar: `c(1, 2, "tres")` produce un vector de texto. Este fenómeno, la coerción (*coercion*), es fuente frecuente de errores silenciosos. Un caso útil: `TRUE` vale `1` y `FALSE` vale `0`, así que `sum()` sobre un vector lógico cuenta cuántos elementos cumplen una condición.

Los elementos pueden llevar nombre, lo que resulta natural para tablas de frecuencia:

```
frec <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)
```

El acceso a los elementos se llama indexación (*indexing*) y se hace con corchetes. R indexa desde 1. Se puede indexar por posición (`frec[3]`), por rango (`frec[2:4]`), por exclusión (`frec[-1]`) o por nombre (`frec["que"]`).

La forma más importante es la indexación lógica (*logical indexing*): se pasa una condición y R devuelve los elementos que la cumplen.

```
frec[frec > 2000]
#>   el   de  que
#> 4521 3890 2103
```

Los operadores de comparación son `==`, `!=`, `>`, `<`, `>=`, `<=` y `%in%` (pertenencia a un conjunto). El error más común de todo el aprendizaje de R es comparar con `=` en lugar de `==`. Las condiciones se combinan con `&` (y), `|` (o) y `!` (negación).

Las operaciones son vectorizadas (*vectorized*): se aplican elemento a elemento sin necesidad de bucles (*loops*). La expresión `(frec_abs / 85000) * 1000` normaliza todas las frecuencias de una vez. Las funciones de resumen (`sum`, `mean`, `max`, `range`) operan sobre el vector completo.

Para ordenar, `sort(frec, decreasing = TRUE)` devuelve el vector ordenado; `order()` devuelve los índices que producirían ese orden, y su utilidad aparecerá al ordenar marcos de datos (*data*

frames), que se verán en la sección 4. Para gráficos rápidos, `barplot(frec)` produce un gráfico de barras a partir de un vector con nombres.

Actividades.

2.1 Crea un vector `pos_freq` con las frecuencias NOUN 1250, VERB 890, ADJ 445, ADV 312, PRON 678. Extrae el tercer elemento, todos menos ADV, y el elemento llamado VERB.

2.2 Obtén con indexación lógica las etiquetas con frecuencia mayor a 500 y cuenta cuántas son usando `sum()`.

2.3 Calcula la frecuencia relativa de cada etiqueta como porcentaje del total, redondeada a dos decimales, y grafícala con `barplot()`.

2.4 Ordena `pos_freq` de mayor a menor y obtén el nombre de la etiqueta más frecuente con `which.max()`.

3. Factores

Un factor es la estructura de R para variables categóricas. Sus categorías posibles son los niveles (*levels*), que de forma predeterminada se ordenan alfabéticamente. Este es el punto donde la teoría de escalas de medición se vuelve código: un factor sin orden es una variable nominal; uno declarado con `ordered = TRUE` es ordinal.

```
pos <- factor(c("NOUN", "VERB", "ADJ", "NOUN", "VERB", "NOUN"))
table(pos)
#>  ADJ NOUN VERB
#>    1    3    2
```

`table()` produce la tabla de frecuencias y, a diferencia de lo que ocurre con texto simple, conserva los niveles con frecuencia cero tras un filtrado; `droplevels()` los elimina si es necesario. El orden de los niveles se controla al crear el factor con el argumento `levels`, lo que determina el orden de las categorías en tablas y gráficos.

Para escalas ordinales se declara el orden explícitamente:

```
acept <- factor(c("marginal", "aceptable", "inaceptable"),
                levels = c("inaceptable", "marginal", "aceptable"),
                ordered = TRUE)
median(acept)
#> [1] marginal
mean(acept)
#> Warning: argument is not numeric or logical
```

Que la mediana funcione y la media produzca una advertencia (*warning*) ilustra, en el propio lenguaje, qué estadísticos son legítimos para cada escala.

Una trampa clásica: `as.numeric()` sobre un factor devuelve los códigos internos de los niveles, no los valores. La conversión correcta es siempre `as.numeric(as.character(x))`.

Cruzar dos factores con `table(f1, f2)` produce una tabla de contingencia; `prop.table()` la convierte en proporciones, con `margin = 1` por filas y `margin = 2` por columnas.

Actividades.

3.1 Crea un factor con los valores pretérito, presente, futuro (repetidos en cualquier combinación de seis elementos). Reordena los niveles según la secuencia temporal y compara el `barplot(table(...))` antes y después.

3.2 Construye un factor ordinal con niveles A1 a C2 del MCER para ocho aprendientes. Calcula la mediana y filtra los de nivel B2 o superior.

3.3 Crea `puntos <- factor(c(5, 8, 3, 8, 5))`, aplica `as.numeric()` directamente, observa el error y corrígelo hasta que `sum()` devuelva 29.

4. Marcos de datos (*data frames*)

Un marco de datos es una tabla: cada columna es un vector, todas las columnas tienen la misma longitud, y columnas distintas pueden ser de tipos distintos. Es el formato habitual para metadatos de corpus.

```
meta <- data.frame(
  texto  = c("T01", "T02", "T03", "T04", "T05"),
  genero = c("narrativo", "académico", "narrativo", "periodístico",
  ↪  "académico"),
  tokens = c(1250, 3400, 890, 2100, 4200),
  types  = c(520, 980, 410, 760, 1150)
)
```

Al abrir cualquier conjunto de datos, ejecuta `str()`: muestra dimensiones, nombres, tipos y una muestra de valores. Complementan `dim()`, `head()` y `summary()`.

Una columna se extrae con `meta$tokens` y es un vector ordinario, de modo que todo lo de la sección 2 aplica. La indexación bidimensional usa `[fila, columna]`, con la coma obligatoria: `meta[1, ]` es la primera fila y `meta[, "tokens"]` la columna completa.

Filtrar filas es indexación lógica en la posición de las filas, o bien `subset()`, que evita repetir el nombre del marco de datos:

```
meta[meta$tokens > 2000, ]
subset(meta, genero == "académico")
```

Asignar a una columna inexistente la crea: `meta$ttr <- round(meta$types / meta$tokens, 3)`. Para ordenar se usa `order()`: `meta[order(-meta$tokens), ]` ordena por tokens en forma descendente. Para calcular estadísticos por grupo, `aggregate(tokens ~ genero, data = meta, FUN = mean)`; la notación `y ~ x` se lee "y en función de x" y reaparecerá en los modelos estadísticos del módulo.

El intercambio de datos se hace en formato CSV. Al guardar, incluye `row.names = FALSE` para evitar una columna espuria; con tildes y eñes conviene fijar `fileEncoding = "UTF-8"`. Los archivos generados con configuración regional en español suelen usar punto y coma como separador: `read.csv("archivo.csv", sep = ";", dec = ",")`. Ten presente que desde R 4.0 el texto ya no se convierte en factor de forma automática al crear o leer marcos de datos, a diferencia de lo que asume la bibliografía anterior a 2020, incluido Gries.

Para apilar filas de dos tablas con las mismas columnas se usa `rbind()`; para unir dos tablas por una columna común, `merge(x, y, by = "texto")`, que de forma predeterminada conserva solo las filas presentes en ambas.

Actividades.

4.1 Crea un marco de datos `hablantes` con seis filas y columnas id, edad, sexo, ciudad y sujeto_nulo (proporción entre 0 y 1). Inspecciónalo con `str()`.

4.2 Filtra los menores de 40 años, primero con indexación lógica y luego con `subset()`.

4.3 Añade una columna grupo_etario que valga "joven" bajo los 40 años y "mayor" en caso contrario, usando `ifelse()` (sección 6).

4.4 Calcula la media de sujeto_nulo por ciudad con `aggregate()`. Guarda el marco de datos como CSV, vuelve a cargarlo y verifica con `str()` que los tipos se conservaron.

5. Listas

Una lista puede contener objetos de cualquier tipo y longitud, incluidas otras listas o marcos de datos completos. Su relevancia práctica es doble.

Primero, los resultados de las pruebas estadísticas son listas, y saber recorrerlas da acceso a componentes que la salida no muestra:

```
resultado <- t.test(x, y)
names(resultado)
resultado$p.value
```

Segundo, cuando una función propia debe devolver varias cosas, la lista es la solución natural.

La distinción central: el corchete simple `[ ]` devuelve una sublista; el doble `[[ ]]` devuelve el contenido del elemento. La notación `$nombre` equivale al doble corchete. Como los elementos son objetos completos, la indexación se encadena: `analisis$frecuencias["que"]`.

Para aplicar una función a cada elemento sin escribir bucles se usan `lapply()` (devuelve una lista) y `sapply()` (simplifica a vector cuando es posible):

```
textos <- list(t1 = c("el", "gato", "come"), t2 = c("la", "casa", "es",
↪  "grande"))
sapply(textos, length)
#> t1 t2
#>  3  4
```

Actividades.

5.1 Ejecuta un `t.test()` entre dos vectores numéricos inventados y extrae de la lista resultante el valor p (*p-value*) y el estadístico t.

5.2 Escribe una función `describir()` que reciba un vector numérico y devuelva una lista con media, mediana y número de elementos.

5.3 Crea una lista con cuatro vectores de palabras de distinta longitud y obtén con `sapply()` el número de ocurrencias de cada uno.

6. Programación elemental

Un condicional `if` ejecuta código cuando se cumple una condición, con alternativas mediante `else` y `else if`. Evalúa un solo valor: sobre vectores produce un error. Para condiciones elemento a elemento se usa `ifelse(condición, valor_si, valor_no)`, la forma habitual de crear variables categóricas a partir de numéricas. Con más de dos categorías, `cut()` divide un vector numérico en intervalos etiquetados y devuelve directamente un factor.

Un bucle (*loop*) `for` repite un bloque para cada elemento de un vector:

```
for (p in c("casa", "sol")) {
  cat(p, "tiene", nchar(p), "caracteres\n")
}
```

Cuando el bucle produce resultados, crea el contenedor antes (`res <- numeric(n)`) y recorre con `seq_along()`. Pero la regla general es otra: si existe una versión vectorizada, úsala. El ejemplo anterior se resuelve mejor con `nchar(palabras)` directamente. Reserva los bucles para procesos que dependen del paso anterior o que leen varios archivos. El bucle `while` repite mientras una

condición sea verdadera y se usa cuando el número de iteraciones no se conoce de antemano; verifica siempre que algo dentro del cuerpo modifique la condición, o el bucle no terminará.

La búsqueda de patrones en texto, instrumento central de la lingüística de corpus, se hace con `grep()` y `grepl()`:

```
corpus <- c("El gato duerme", "La casa grande", "Los gatos son
↪  independientes")
grepl("gat", corpus)
#> [1]  TRUE FALSE  TRUE
grep("gat", corpus, value = TRUE)
#> [1] "El gato duerme"              "Los gatos son independientes"
```

`grepl()` devuelve valores lógicos y `grep()` posiciones, o los elementos mismos con `value =` TRUE. Ambas aceptan expresiones regulares (*regular expressions*) y el argumento `ignore.case =` TRUE.

Tres hábitos desde el principio. Comenta el porqué de las decisiones, no lo que el código ya dice. Usa nombres descriptivos: el código se escribe una vez y se lee muchas. Y fija la semilla (*seed*) con `set.seed()` cada vez que haya azar, porque sin ella el análisis no es reproducible.

Sobre los errores: los más frecuentes son el objeto inexistente por error de tipeo, el paréntesis sin cerrar (R queda esperando con el símbolo +) y la función mal escrita. Leer el mensaje completo resuelve la mayoría de los casos; buscar el mensaje literal en internet es práctica normal, no un atajo.

Actividades.

6.1 Escribe una función `evaluar_ttr()` que devuelva “alta” si la razón tipo-ocurrencia (*type-token ratio*) supera 0.5, “media” entre 0.3 y 0.5, y “baja” bajo 0.3. Aplícala al vector `c(0.62, 0.41, 0.22)` con `sapply()`.

6.2 Dado `frec <- c(5, 120, 45, 890, 12, 340)`, clasifica cada valor con `cut()` en tres niveles: baja (hasta 50), media (51 a 300) y alta (sobre 300). Verifica que el resultado es un factor.

6.3 Con el vector `corpus` de esta sección, obtén las oraciones que contienen alguna palabra que empiece por “gat” y cuenta cuántas son.

6.4 Escribe una función `perfil()` que reciba un vector de oraciones y devuelva una lista con el número de oraciones, el total de ocurrencias (separando por espacios con `strsplit()`) y la longitud media por oración.

Referencias

Gries, S. Th. (2009). *Quantitative Corpus Linguistics with R*. Routledge. Capítulo 3. Wickham, H. y Grolemund, G. (2017). *R for Data Science*. O’Reilly. Disponible en r4ds.hadley.nz. Levshina, N. (2015).

How to Do Linguistics with R. John Benjamins.